THE CYBERDECK BUILDER'S MANUAL

Module 1 of 5 — Linux & Command Line Foundations

A self-paced, hands-on curriculum for designing and building a custom cyberdeck with ethical hacking / security-research capability — one skill module at a time, each one earned before the next unlocks.

How this manual works

This is the first of several modules. Each module teaches the skills the *next* module assumes you already have — nothing here is filler.

Work through the lessons in order. Do the exercises for real, on real hardware if you can get your hands on a Pi — reading alone won't build the muscle memory.

Each module ends with a Checkpoint. If you can complete the checkpoint without looking anything up, you're ready for the next module. If not, that's fine — repeat the lessons that felt shaky.

Tell me when you've finished this module (or when you're stuck) and I'll build Module 2: 3D Design & the Physical Build, tailored to what you actually know by then.

The Full Roadmap

Five modules, each building toward the final integrated cyberdeck. You only get the next module's full content once you're ready — this page is just so you can see where Module 1 fits into the bigger picture.

#	Module	What it gives you
1	**Linux & Command Line Foundations**	Comfortable on a headless Raspberry Pi over SSH: filesystem, permissions, packages, proces
2	**3D Design & the Physical Build**	CAD fundamentals (FreeCAD), designing a simple printable enclosure, 3D printing basics, goi
3	**Electronics & Power**	Reading a datasheet, basic soldering, breadboarding, battery safety (LiPo), power delivery (UP
4	**RF, Wireless & Security Tooling**	How sub-GHz / NFC / IR / WiFi actually work, ESP32 firmware (e.g. Marauder/Bruce), legal &
5	**Integration & Software**	Wiring all subsystems into one deck, a clean OS image, your own launcher/dashboard, and do

You are here: Module 1. Why start with the command line instead of the fun stuff? Because every later module assumes you can SSH into a Pi, find a file, fix a permission error, install a package, and not panic when something prints a wall of red text. Builders who skip this step end up stuck on basic Linux problems halfway through a 3D-printing or RF lesson, which is a frustrating place to learn it.

Module 1: Linux & Command Line Foundations

By the end of this module you'll be able to take a bare Raspberry Pi (or any similar single-board computer), get it running headless over SSH, navigate and manage its filesystem confidently, install and update software, manage permissions and running processes, check its network state, and write a small bash script to automate a task. These are the exact skills every cyberdeck build log assumes you already have.

What you'll need

- A Raspberry Pi — a Pi 4 or Pi 5 (any RAM size) is ideal, but even an old Pi 3B or a Pi Zero 2 W works fine for this module. You're learning Linux, not benchmarking it.
- A microSD card, 16–32GB is plenty (Class 10 / A1 rated).
- A USB-C (or micro-USB, depending on Pi model) power supply.
- A computer to flash the SD card and SSH from — your normal laptop is fine.
- Optional but handy: a spare HDMI cable and a keyboard, only for the very first boot if the headless setup trips you up and you need to see what's happening directly.

> **No Raspberry Pi yet?**
> You can still do almost all of Lessons 1.4–1.10 using a Linux virtual machine (e.g. VirtualBox + Ubuntu Server, or WSL2 on Windows) or even an Otago Polytechnic lab machine if it runs Linux. The command line skills transfer directly. Just come back to Lessons 1.1–1.3 (flashing & first boot) once you've got the actual Pi in hand.

Lesson 1.1 — Why Linux, and Picking a Distro

Almost every cyberdeck, and nearly every piece of security/RF tooling you saw in those build logs (Flipper firmware tooling, ESP32 flashing tools, SDR software, network scanners), is built for or runs natively on Linux. Windows can do some of it, but you'll constantly be fighting the tooling instead of learning it. Three reasons Linux wins here:

- **It's what the hardware expects.** Raspberry Pi OS, Arch Linux ARM, and most SBC images are Linux. The Pi in the cyberdeck video booted Arch — that's a deliberate, fairly advanced choice; you'll start somewhere friendlier.
- **It's transparent.** Everything is a file or a process you can inspect. There's no hidden registry or background service you can't see — useful when you're debugging a custom-built device with no manual.
- **It's the lingua franca of security tooling.** Kali, the Flipper companion tools, most SDR and wireless tooling — built Linux-first.

For this module, use **Raspberry Pi OS Lite (64-bit)** if you're on real Pi hardware — it's Debian-based, has the largest community, and the "Lite" version skips the desktop GUI, which is exactly what you want: a cyberdeck talks to you over SSH or a small display, not a full desktop. If you're doing this in a VM instead, Ubuntu Server is the closest equivalent.

Lesson 1.2 — Flashing the OS & Headless Setup

"Headless" means no monitor or keyboard plugged into the Pi — you control it entirely over the network via SSH. This is how every cyberdeck in those build logs actually gets worked on during development, even if it later gets its own screen.

Step by step:

- Download and install the official **Raspberry Pi Imager** on your laptop (raspberrypi.com/software).
- Insert your microSD card into your laptop (use a USB adapter if needed).
- Open Imager → choose OS: *Raspberry Pi OS (other)* → *Raspberry Pi OS Lite (64-bit)*.
- Choose your SD card as the storage target.
- **Before clicking write**, click the gear/settings icon (or it may prompt you automatically) to open Advanced Options. This is the step that makes it headless-capable:
- Set a hostname (e.g. `cyberdeck`) so you can find it on the network by name.
- Enable SSH, and choose "Use password authentication" for now (we'll cover SSH keys properly later in this lesson).
- Set a username and a strong password — do NOT leave it as the old default pi/raspberry, that default was removed for security reasons and recent images force you to set your own.
- Configure your WiFi SSID, password, and country (NZ) so it joins your network on first boot.

Write the image, eject the card safely, put it in the Pi, and power it on. Give it 60–90 seconds for first boot.

Lesson 1.3 — First Connection Over SSH

From your laptop's terminal (Terminal on Mac/Linux, or PowerShell / Windows Terminal on Windows 10+, which has SSH built in):

```
ssh username@cyberdeck.local
# replace 'username' with what you set in Imager,
# and 'cyberdeck' with the hostname you chose
```

If `.local` mDNS resolution doesn't work on your network (common on some routers or NZ ISP-supplied modems), find the Pi's IP address instead by checking your router's connected-devices list, or running a network scan from your laptop:

```
# on macOS/Linux
arp -a | grep -i b8:27:eb   # common Pi MAC prefix, or just eyeball hostnames

# or use a network scanner
nmap -sn 192.168.1.0/24     # adjust to match your home subnet
```

Then connect with `ssh username@<the IP you found>`. First time connecting, you'll get a fingerprint warning — type `yes`, this is normal and expected on a first connection.

You're now in a remote shell running on the Pi itself. Everything you type executes on that little board, not your laptop. Sit with that for a second — this is the core working pattern for the entire cyberdeck project.

> **A note on security hygiene (relevant given where this project is headed)**
>
> Password-based SSH is fine for learning, but before you start adding any RF/pentest tooling in later modules, you'll want to switch to SSH key authentication and disable password login entirely. We'll do that properly in Module 4, but it's worth knowing now: a cyberdeck full of hacking tools that's still wide open on default-password SSH is a genuinely bad look, and a real risk if it's ever on an untrusted network.

Exercise 1.3

- Successfully SSH into your Pi from your laptop.
- Run `whoami` and `hostname` — confirm they show your username and chosen hostname.
- Disconnect with `exit`, then reconnect without looking at your notes.

Lesson 1.4 — The Filesystem & Navigation

Linux has one unified filesystem tree starting at `/` (called "root", not to be confused with the root user). Everything — disks, USB devices, even hardware like your future ESP32 module — shows up somewhere in this tree.

Command	What it does
`pwd`	Print working directory — where am I right now?
`ls`	List files in the current directory. Add -la for hidden files + details.
`cd <path>`	Change directory. cd .. goes up one level. cd ~ goes home.
`mkdir <name>`	Make a new directory.
`touch <name>`	Create an empty file (or update its timestamp).
`cp <src> <dst>`	Copy a file. Add -r to copy a whole directory.
`mv <src> <dst>`	Move or rename a file/directory.
`rm <name>`	Delete a file. Add -r for a directory. There is no undo — no Recycle Bin.
`cat <file>`	Print a file's full contents to the screen.
`less <file>`	View a file page-by-page (q to quit). Better for long files than cat.
`man <command>`	Open the manual page for any command — your built-in reference.

One key mental model: **absolute vs relative paths.** `/home/username/Documents` is absolute — it always means the same place no matter where you currently are. `Documents` or `../Documents` is relative — it depends on your current directory. Get comfortable reading both.

Exercise 1.4

```
mkdir ~/cyberdeck-notes
cd ~/cyberdeck-notes
touch readme.txt
echo "Module 1 in progress" > readme.txt
cat readme.txt
cp readme.txt readme-backup.txt
ls -la
```

Type each line yourself (don't paste) and predict what each command will print before you press enter. The `echo "text" > file` pattern writes text into a file — you'll use this constantly later for config files.

Lesson 1.5 — Permissions & sudo

Every file and directory has an owner, a group, and a set of permissions for read/write/execute, separately for the owner, the group, and everyone else. Run `ls -l` in any directory and you'll see something like:

```
-rw-r--r-- 1 kelly kelly  142 Jun 30 10:02 readme.txt
drwxr-xr-x 2 kelly kelly 4096 Jun 30 10:01 cyberdeck-notes
```

Reading that string left to right: the first character is the type (`-` = file, d = directory). Then three groups of three: owner permissions, group permissions, everyone-else permissions, each as read (r), write (w), execute (x). So `rw-r--r--` means the owner can read+write, everyone else can only read.

`chmod` changes permissions, `chown` changes the owner. You'll use chmod constantly later when you write your own scripts — a script won't run until you mark it executable:

```
chmod +x myscript.sh    # add execute permission for everyone
chmod 755 myscript.sh   # explicit: owner rwx, group r-x, others r-x
```

sudo ("superuser do") temporarily runs a single command with administrator privileges. You'll need it for installing software, editing system config files, and later, for a lot of the RF/hardware tooling that needs direct hardware access. Use it deliberately, not reflexively — if a command fails, understand *why* it needed elevated permission before just slapping sudo in front of everything.

> **Why this matters more for you than the average Linux beginner**
>
> A lot of the wireless/RF tooling you'll add in Module 4 (packet capture, BadUSB, raw radio access) requires elevated permissions because it talks directly to hardware. Understanding permissions properly now means that later, when something needs sudo, you'll understand *why* instead of just pattern-matching "add sudo until it works", which is how people accidentally run security tools with far more access than intended.

Lesson 1.6 — Package Management

Raspberry Pi OS uses **APT** (Debian's package manager). This is how you'll install literally everything from here on — CAD tools, Python libraries, firmware flashers, SDR software.

```
sudo apt update          # refresh the list of available package versions
sudo apt upgrade         # actually install any newer versions of installed packag
sudo apt install git     # install a new package (here, git)
sudo apt remove git      # uninstall it
apt search keyword       # search for a package by name/description
```

Always run `update` before `install` on a freshly flashed Pi — the image is built at some point in the past and the package index it ships with is stale on day one.

Keeping a system patched matters more once it's running anything network-facing (which a cyberdeck, by definition, is) — unpatched, known vulnerabilities are the single most common way small Linux devices get compromised on a network, full stop.

Lesson 1.7 — Editing Files in the Terminal

You'll constantly need to edit config files over SSH where there's no graphical text editor. **nano** is the easiest place to start — it's pre-installed and shows its shortcuts on screen.

```
nano somefile.txt
# Ctrl+O then Enter = save ("Write Out")
# Ctrl+X = exit
# Ctrl+K = cut a line, Ctrl+U = paste it
```

Once you're comfortable, it's worth learning basic **vim** too — it's the default editor on most minimal Linux installs you'll encounter (including some firmware build environments later on), but don't let it block you now. nano is a completely fine permanent choice if you prefer it.

Lesson 1.8 — Processes & systemd

Every running program is a **process** with a process ID (PID). Your finished cyberdeck will eventually run several services constantly in the background (a display service, network tools, maybe a custom dashboard) — this is the lesson that demystifies how that works.

Command	What it does
`ps aux`	List all running processes system-wide.
`top / htop`	Live, updating view of processes and resource usage (htop is nicer, install with apt).
`kill <PID>`	Politely ask a process to stop.
`kill -9 <PID>`	Forcefully terminate it (last resort).
`command &`	Run a command in the background, freeing up your terminal.
`systemctl status <name>`	Check whether a background service is running.
`sudo systemctl enable <name>`	Make a service start automatically on every boot.

`systemd` is the system that manages background services ("daemons") and starts them on boot. In Module 5, your custom dashboard software will almost certainly run as a systemd service so it launches automatically the instant the deck powers on, with no manual login required — exactly how the deck in the video just "works" the second you flip the power switch.

Lesson 1.9 — Basic Networking From the CLI

```
ip a                # show network interfaces and their IP addresses
ping google.com     # test connectivity (Ctrl+C to stop)
hostname -I         # quick way to see your own IP address(es)
curl ifconfig.me    # see your public IP, as the internet sees you
```

This is foundational for Module 4: every RF/WiFi security tool fundamentally works by reading and writing at this network-interface level, just with more specialised tools on top. `ip a` showing you `wlan0` (WiFi) and `eth0` (Ethernet) as named interfaces is the same concept that later lets an ESP32 module register itself as an interface you can drive from the command line.

Lesson 1.10 — Your First Bash Script

A shell script is just a text file of commands you'd otherwise type one by one. This is the automation skill that, later, becomes the script that flashes your ESP32, or the boot-time script that starts your dashboard, or the one-liner that backs up your CAD files before a print. Create a file called `system-check.sh`:

```
#!/bin/bash
# A simple system-check script

echo "=== Cyberdeck System Check ==="
echo "Hostname: $(hostname)"
echo "Uptime:   $(uptime -p)"
echo "Disk usage:"
df -h / | tail -n 1
echo "Memory usage:"
free -h | grep Mem
echo "IP address: $(hostname -I)"
echo "=== Check complete ==="
```

Then make it executable and run it:

```
chmod +x system-check.sh
./system-check.sh
```

Notice the pattern: `$(command)` runs a command and drops its output right into your text. The `#!/bin/bash` on line one is called a "shebang" — it tells the system which interpreter to run the script with. Every script you write from here on starts with one of these.

Exercise 1.10

- Get system-check.sh running and producing real output from your actual Pi.
- Modify it to also print the current date/time and the Pi's CPU temperature (hint: `vcgencmd measure_temp` on Raspberry Pi OS).
- Add a line that only prints a warning if free disk space drops below some threshold — you don't need to get this perfect, just attempt an if-statement and look up the bash syntax for it. Productive struggle is the point.

Module 1 Checkpoint

Before moving on to Module 2 (3D Design & the Physical Build), make sure you can do every one of these, ideally without notes:

- Flash a fresh SD card with headless SSH + WiFi configured, from scratch, in under 10 minutes.
- SSH into the Pi and find its IP address two different ways if hostname resolution fails.
- Create, move, copy, and delete files and directories confidently, understanding the difference between absolute and relative paths.
- Read a permissions string (e.g. rwxr-xr-x) and explain what it means.
- Use sudo correctly and explain, in your own words, why it's needed for a given command.
- Update the system and install a new package with apt.
- Edit a file in nano without help.
- Find and stop a running process by PID.
- Read the output of `ip a` and identify your WiFi interface and current IP address.
- Write, save, permission, and run a basic bash script that uses at least one variable and one command substitution.

If all of that feels solid: you're done with Module 1. Let me know and I'll build Module 2 — 3D Design & the Physical Build — covering FreeCAD fundamentals, designing your first printable part, and the practical realities of 3D printing (materials, tolerances, what actually goes wrong) before you touch the cyberdeck's enclosure design itself.

If something felt shaky: that's completely normal for a first pass at the command line. Tell me which lesson didn't click and I'll re-explain it a different way, or build you a few extra targeted exercises, rather than ploughing ahead onto skills that build on a shaky foundation.
